Python Basics

15 Practice Problems

Variables · Operators · Loops · Lists · Tuples · Dicts · Sets · Strings

1. Age Calculator

Tags: Variables Input / Output Operators

Description

Ask the user to enter their name and birth year.
Calculate their age and display a personalised greeting message.

Constraints

- `name must be a non-empty string`
- `birth year must be a valid integer`

Note: *Assume the current year is 2025.*

Example

Input:

```
Enter your name: Alice
Enter your birth year: 2000
```

Output:

```
Hello Alice, you are 25 years old!
```

---

2. Number Analyser

Tags: Input / Output Operators Conditionals

Description

Ask the user to enter 3 numbers.
Display the following:

- Sum of all 3 numbers
- Product of all 3 numbers
- Average of all 3 numbers
- Which number is the largest

Constraints

- `All inputs are valid integers or floats`

Example

Input:

```
Enter number 1: 4
Enter number 2: 7
Enter number 3: 2
```

Output:

```
Sum:     13
Product: 56
Average: 4.33
Largest: 7
```

---

3. Word Inspector

Tags: Strings Input / Output

Description

Ask the user to enter any word.
Display the following:

- The word in UPPERCASE
- The word in lowercase
- The word reversed
- Total number of characters

Constraints

- `Input will be a single word with no spaces`

Example

Input:

```
Enter a word: Python
```

Output:

```
Uppercase:  PYTHON
Lowercase:  python
Reversed:   nohtyP
Characters: 6
```

---

4. Multiplication Table

Tags: Loops Input / Output Operators

Description

Ask the user to enter a number.

Print the full multiplication table for that number from 1 to 10.

Constraints

- Input will be a positive integer

Example

Input:
```
Enter a number: 5
```

Output:
```
5 x 1  =  5
5 x 2  =  10
5 x 3  =  15
...
5 x 10 =  50
```

---

5. Fruit List Sorter

Tags: Lists Loops Input / Output

Description

Ask the user to enter 5 fruit names one by one and store them in a list.
After all 5 fruits are entered display:

- All fruits as entered
- The list sorted A to Z
- The list sorted Z to A

Constraints

- Exactly 5 fruit names must be entered
- Inputs are non-empty strings

Example

Input:
```
Fruit 1: Mango
Fruit 2: Apple
Fruit 3: Banana
Fruit 4: Cherry
Fruit 5: Grape
```

Output:
```
As entered: ['Mango', 'Apple', 'Banana', 'Cherry', 'Grape']
A to Z:     ['Apple', 'Banana', 'Cherry', 'Grape', 'Mango']
Z to A:     ['Mango', 'Grape', 'Cherry', 'Banana', 'Apple']
```

---

6. Divisible by 3

Tags: Loops Conditionals Input / Output

Description

Ask the user to enter a number n.
Print all numbers from 1 to n that are divisible by 3.

Constraints

- `n must be a positive integer`

Example

Input:

```
Enter a number: 20
```

Output:

```
Numbers divisible by 3:
3   6   9   12   15   18
```

7. Sentence Stats

Tags: Strings Input / Output Operators

Description

Ask the user to enter a sentence.
Display the following:

- Total number of words
- Total characters excluding spaces
- The sentence in UPPERCASE

Constraints

- `Input will contain at least one word`

Example

Input:

```
Enter a sentence: Python is fun to learn
```

Output:

```
Word count:               5
Characters (no spaces):   18
Uppercase:                PYTHON IS FUN TO LEARN
```

8. List Statistics

Tags: Lists Loops Input / Output Operators

Description

Ask the user to enter 5 numbers one by one and store them in a list.
Display the following:

- All 5 numbers
- The largest number
- The smallest number
- The sum of all numbers

Constraints

- `All inputs are valid integers or floats`

Example

Input:

```
Number 1: 12
Number 2: 45
Number 3: 7
Number 4: 30
Number 5: 21
```

Output:

```
Numbers: [12, 45, 7, 30, 21]
Largest:  45
Smallest: 7
Sum:      115
```

9. Country Lister

Tags: Tuples Loops

Description

Use the tuple below in your program.
Loop through it and print each country with its position number.

Data

```
countries = ("USA", "Canada", "Brazil", "Germany", "Japan")
```

Constraints

- `Do not modify the tuple`
- `Position numbers start from 1`

Example

Input:
```
(No user input needed — use the tuple above)
```

Output:
```
1.  USA
2.  Canada
3.  Brazil
4.  Germany
5.  Japan
```

---

10. Number Collector

Tags: Lists Loops Conditionals Input / Output

Description

Ask the user to keep entering numbers one by one.
Stop when the user types 0.
After stopping display:

- All even numbers from the list
- All odd numbers from the list
- The sum of all entered numbers (excluding 0)

Constraints

- ▸ At least one number must be entered before 0
- ▸ 0 is not included in the sum or lists

Example

Input:
```
Enter number (0 to stop): 4
Enter number (0 to stop): 7
Enter number (0 to stop): 12
Enter number (0 to stop): 9
Enter number (0 to stop): 0
```

Output:
```
Even: [4, 12]
Odd:  [7, 9]
Sum:  32
```

---

11. Capital Finder

Tags: Dictionaries Conditionals Strings Input / Output

Description

Use the country-capital data below and store it in a dictionary.

Ask the user to type a country name.

- If found, display its capital.
- If not found, display: Country not found.

Data

```
"France"  → "Paris"
"Germany" → "Berlin"
"Japan"   → "Tokyo"
"Brazil"  → "Brasilia"
"India"   → "New Delhi"
```

Constraints

- ▸ `Search is case-insensitive`
- ▸ `Input will be a single word or two-word country name`

Note: *Search must be case-insensitive — 'japan' and 'Japan' both work.*

Example

Input:

```
Enter a country: japan
```

Output:

```
The capital of Japan is Tokyo.
```

---

12. Vowel Counter

Tags: Strings Loops Conditionals Input / Output

Description

Ask the user to enter a sentence.
Count how many times each vowel (a, e, i, o, u) appears.
Display the count for every vowel.

Constraints

- ▸ `Input will contain at least one character`
- ▸ `Count all occurrences including uppercase letters`

Note: *Ignore case — 'A' and 'a' count as the same vowel.*

Example

Input:

```
Enter a sentence: I love programming in Python
```

Output:

```
a: 1
e: 1
i: 2
o: 3
u: 0
```

---

13. Set Operations

Tags: Sets Lists Input / Output

Description

Ask the user to enter two groups of numbers separated by commas.
Convert both groups into sets and display:

- Union — all numbers from both groups combined
- Intersection — numbers that appear in both groups
- Difference — numbers in Group 1 that are NOT in Group 2

Constraints

- `Numbers are separated by commas`
- `All inputs are valid integers`

Example

Input:

```
Group 1: 1, 2, 3, 4, 5
Group 2: 4, 5, 6, 7, 8
```

Output:

```
Union:        {1, 2, 3, 4, 5, 6, 7, 8}
Intersection: {4, 5}
Difference:   {1, 2, 3}
```

---

14. Contact Book

Tags: Dictionaries Loops Input / Output Strings

Description

Build a simple contact book.
Ask the user to enter 4 contacts. For each contact collect:

- Contact name
- Phone number

Store all contacts in a dictionary (name → phone).
After entering, ask the user to search by name.

- If found, display the phone number.

- If not found, display: Contact not found.

Constraints

- `Exactly 4 contacts must be entered`
- `Phone numbers are stored as strings`

Note: *Search must be case-insensitive.*

Example

Input:

```
Name: Alice   Phone: 555-1234
Name: Bob     Phone: 555-5678
Name: Carol   Phone: 555-9012
Name: David   Phone: 555-3456

Search: alice
```

Output:

```
Name:  Alice
Phone: 555-1234
```

---

15. Student Grade Tracker

Tags: Lists Dictionaries Loops Conditionals Input / Output

Description

Ask the user how many students are in the class.
For each student collect their name and score (out of 100).
Store each student as a dictionary and add all to a list.
After all data is entered display:

- Each student's name, score, and result (Passed if score >= 60, else Failed)
- The name and score of the top student
- The class average score

Constraints

- `Number of students >= 1`
- `Scores are integers between 0 and 100`
- `Passing score is 60 or above`

Example

Input:

```
How many students? 3
Student 1 — Name: Alice    Score: 85
Student 2 — Name: Bob      Score: 52
Student 3 — Name: Charlie  Score: 73
```

Output:

```
Alice    |  85  |  Passed
Bob      |  52  |  Failed
Charlie  |  73  |  Passed

Top student:   Alice (85)
Class average: 70.00
```